Introduction to Cybersecurity

Protocols

Jasmin Azemović
CISO | Associate Professor | Microsoft MVP

Protocol

- Human protocols — the rules followed in human interactions
  - Example: Asking a question in class
- Networking protocols — rules followed in networked communication systems
  - Examples: HTTP, FTP, etc.
- Security protocol — the (communication) rules followed in a security application
  - Examples: SSL, IPSec, Kerberos, etc.

Protocols

- Protocol flaws can be very subtle
- Several well-known security protocols have significant flaws
  - Including WEP, GSM, and even IPSec
- Implementation errors can occur
  - Such as IE implementation of SSL
- Not easy to get protocols right…

Ideal Security Protocol

- Satisfies security requirements
  - Requirements must be precise
- Efficient
  - Small computational requirement
  - Small bandwidth usage, network delays…
- Robust
  - Works when attacker tries to break it
  - Works even if environment changes
- Easy to use & implement, flexible…
- Difficult to satisfy all of these!

Entry to military facility

1. Insert badge into reader
2. Enter PIN
3. Correct PIN?

   Yes? Enter

   No? Get shot by security guard

ATM Machine Protocol

1. Insert ATM card
2. Enter PIN
3. Correct PIN?

   Yes? Conduct your transaction(s)

   No? Machine (eventually) eats card

Identify Friend or Foe (IFF)



Russian
MIG

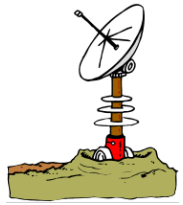
Angola



SAAF
Impala
K

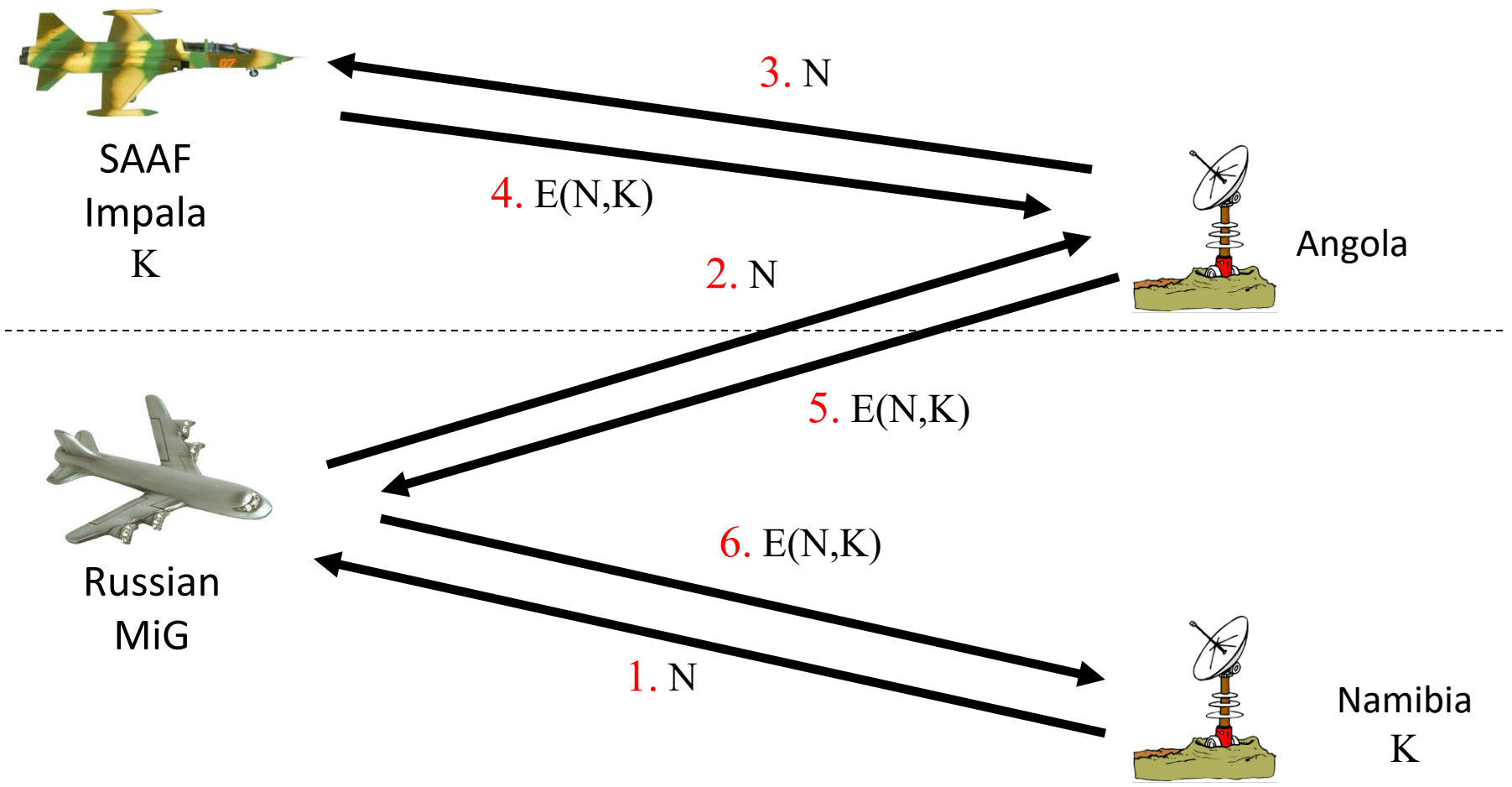
2. E(N,K)

1. N



Namibia
K

MIG in the Middle



Authentication Protocols

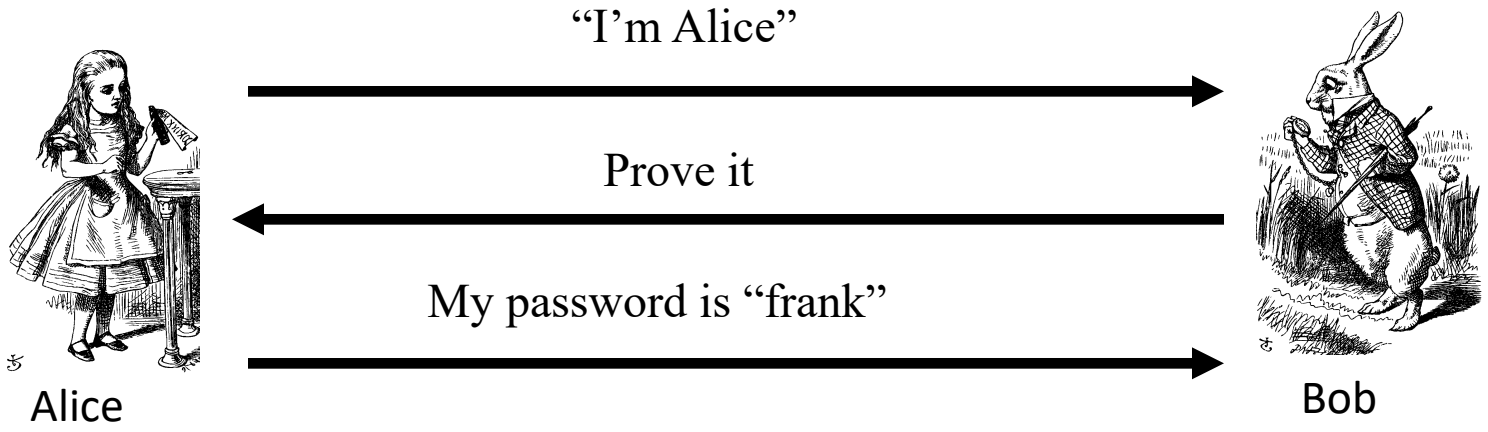
Authentication

- Alice must prove her identity to Bob
  - Alice and Bob can be humans or **computers**
- May also require Bob to prove he's Bob (mutual authentication)
- Probably need to establish a **session key**
- May have other requirements, such as
  - Use public keys
  - Use symmetric keys
  - Use hash functions
  - Anonymity, plausible deniability, etc., etc.

Authentication

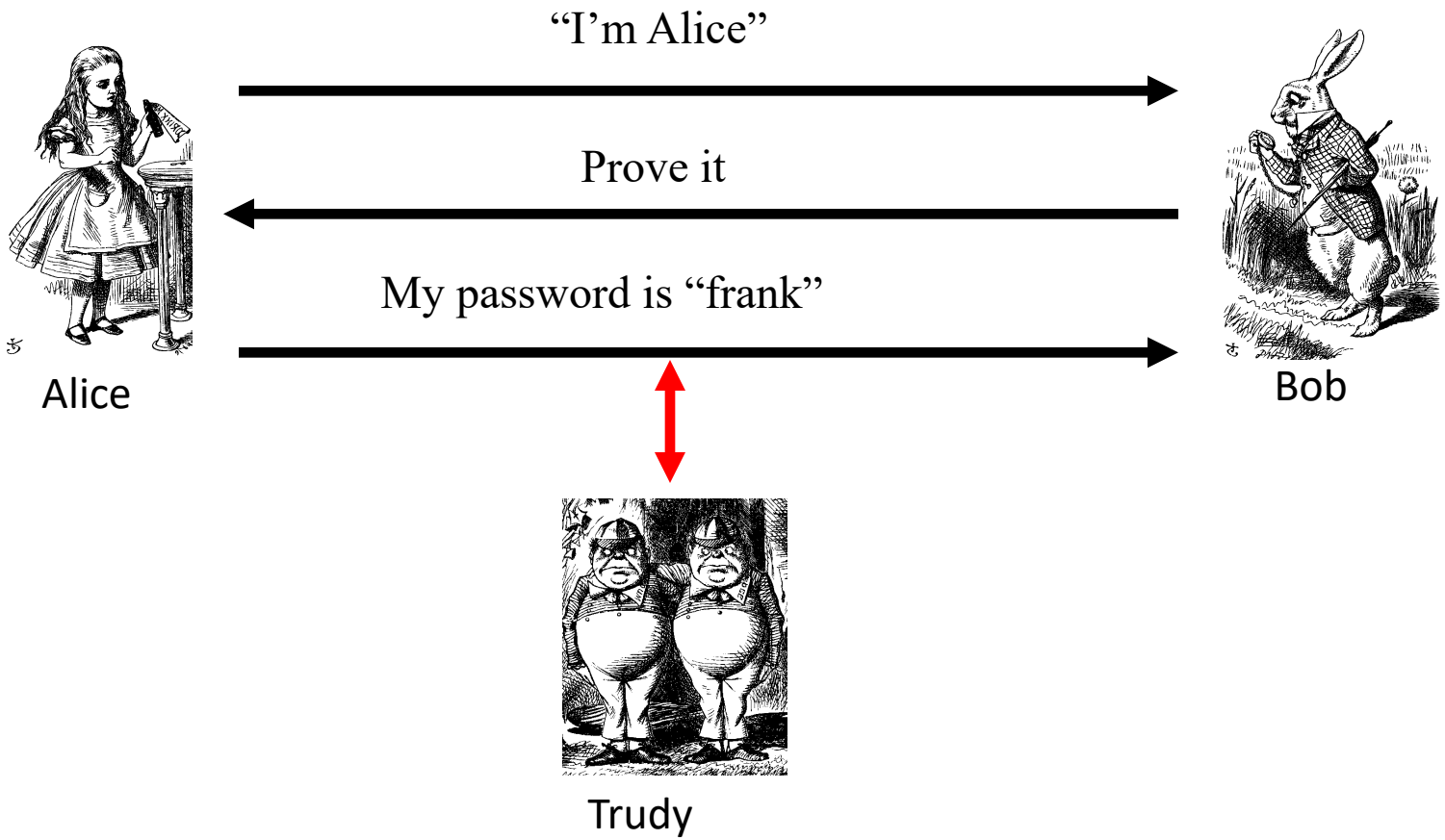
- Authentication on a stand-alone computer is relatively simple
  - Hash password with salt, etc.
  - “Secure path,” attacks on authentication software, keystroke logging, etc., are issues
- Authentication over a network is challenging
  - Attacker can passively observe messages
  - Attacker can replay messages
  - Active attacks possible (insert, delete, change)

Simple Authentication

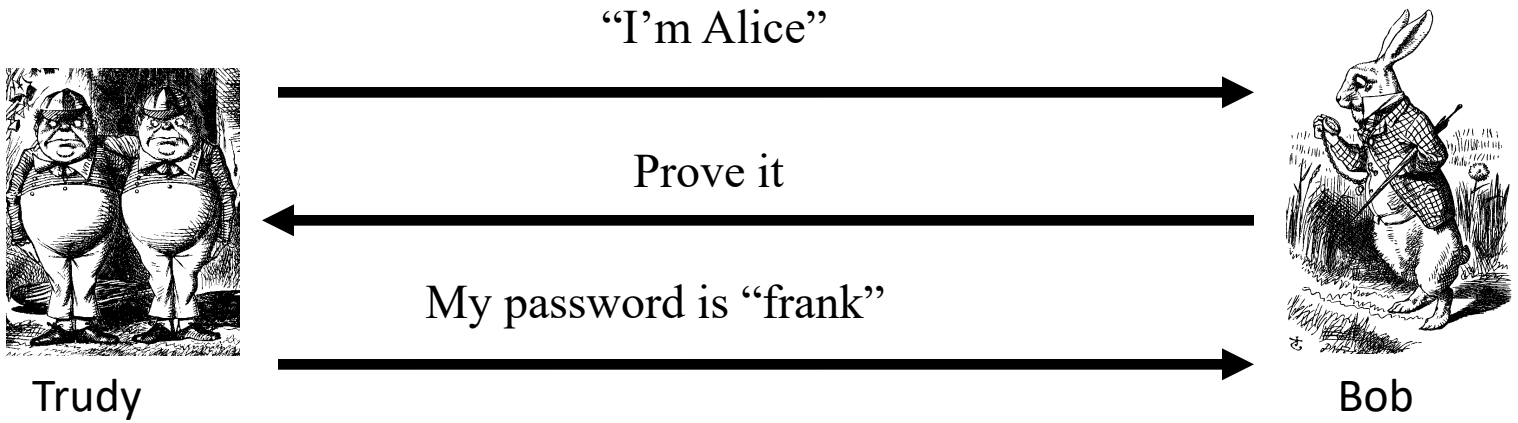


- Simple and may be OK for standalone system
- But insecure for networked system
  - Subject to a **replay** attack (next 2 slides)
  - Also, Bob must know Alice's password

Authentication Attack



Authentication Attack



- This is an example of a **replay** attack
- How can we prevent a replay?

Simple Authentication



Alice

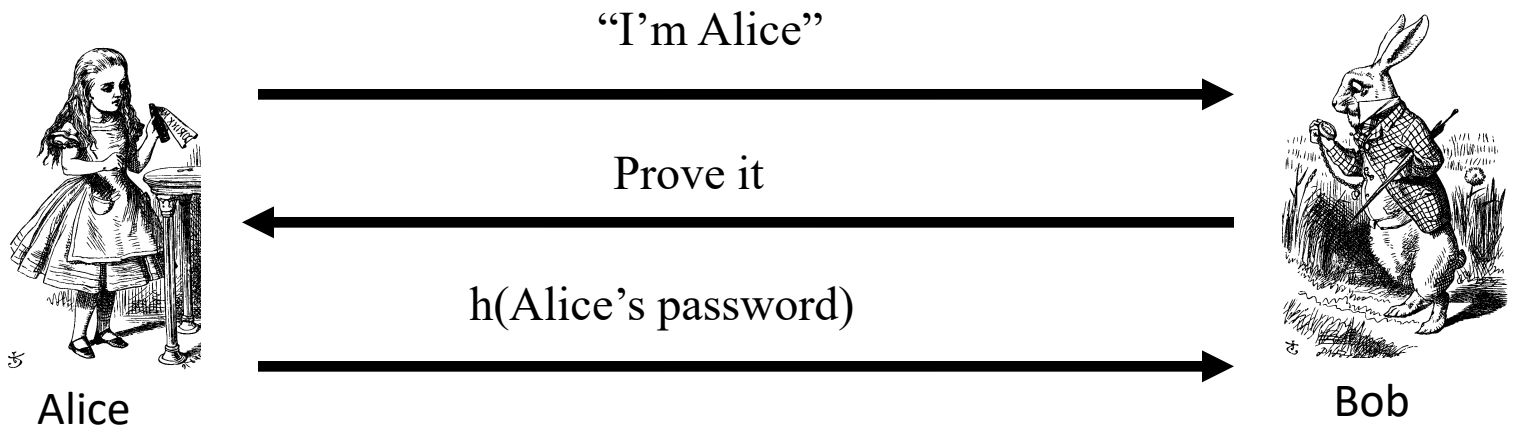
I'm Alice, my password is "frank"



Bob

- More efficient, but…
- … same problem as previous version

Better Authentication



- Better since it hides Alice's password
  - From both Bob and Trudy
- But still subject to replay

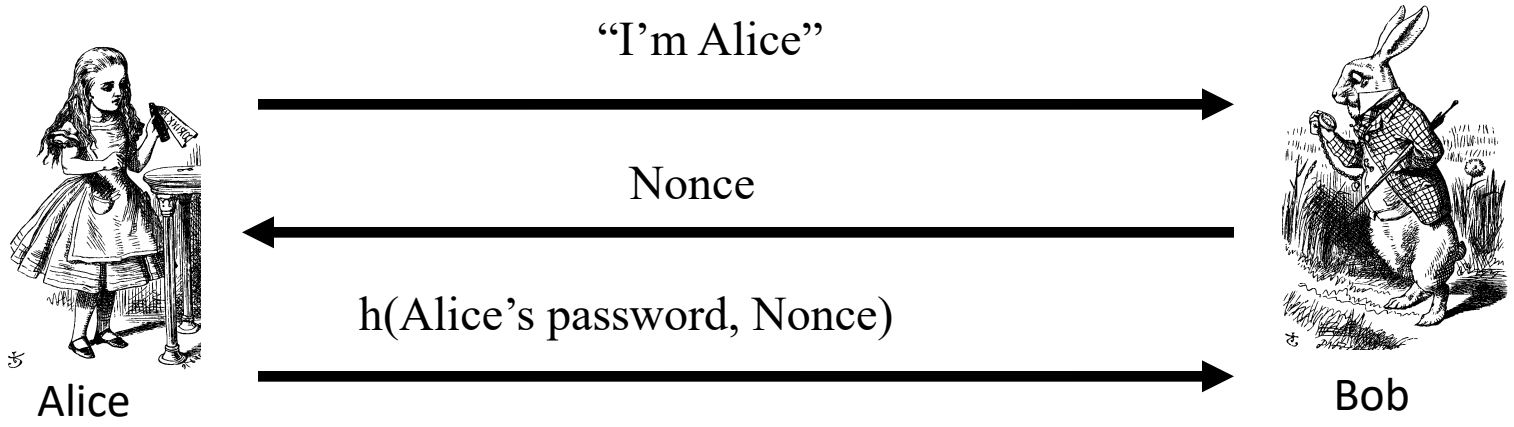
Challenge-Response

- To prevent replay, use ***challenge-response***
  - Goal is to ensure “freshness”
- Suppose Bob wants to authenticate Alice
  - ***Challenge*** sent from Bob to Alice
- Challenge is chosen so that
  - Replay is not possible
  - Only Alice can provide the correct ***response***
  - Bob can verify the response

Nonce

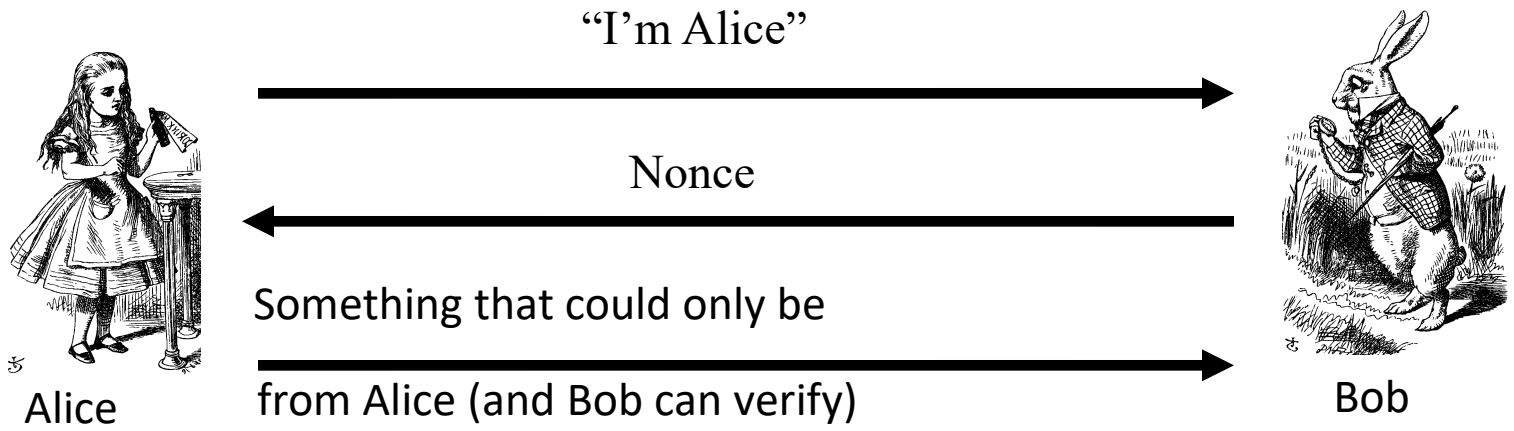
- To ensure freshness, can employ a **nonce**
  - Nonce == **n**umber used **once**
- What to use for nonces?
  - That is, what is the challenge?
- What should Alice do with the nonce?
  - That is, how to compute the response?
- How can Bob verify the response?
- Should we rely on passwords or keys?

Challenge-Response



- Nonce is the challenge
- The hash is the response
- Nonce prevents replay, ensures freshness
- Password is something Alice knows
- Bob must know Alice's pwd to verify

Generic Challenge-Response



- In practice, how to achieve this?
- Hashed pwd works…
- Encryption is better here (Why?)

